Rapport de performance — avant / après optimisation

Date : 4 novembre 2026. Objectif du cahier des charges : Lighthouse ≥ 80 sur Performance, Accessibilité, Bonnes pratiques et SEO ; LCP, CLS, INP mesurés ; temps de réponse API (k6).

⚠️ Ce qui a été mesuré et ce qui ne l'a pas été

Mesure	Statut
Poids JS/CSS/images transférés, LCP, CLS, interaction (proxy INP)	✅ **mesurés** avec Playwright + Chromium sur le banc de prévisualisation (voir `../TEST-REPORT.md`), avec limitation réseau et CPU
Scores Lighthouse	⏳ **non mesurés** (Lighthouse non installable dans l'environnement)
INP réel (Chrome UX / terrain)	⏳ non mesuré — seul un *proxy* (durée maximale d'un événement d'interaction en laboratoire) est fourni
Temps de réponse API avec MongoDB, charge k6	⏳ **non mesurés** — les temps mesurés sur le banc utilisent des données en mémoire et ne sont **pas** représentatifs

Les mesures « avant / après » ci-dessous comparent deux versions du **même frontend** servies par le même banc local : - **Avant** : un seul bundle JavaScript (aucun découpage), non minifié, images de couverture en PNG. - **Après** : découpage par route (`React.lazy`) + Leaflet chargé à la demande, minification, couvertures WebP compressées (sharp), réservation de hauteur pour éviter les décalages.

Conditions : Chromium headless 1280×800, latence 150 ms, débit 1,6 Mb/s descendant / 750 kb/s montant, CPU ralenti ×4 (proche du profil mobile de Lighthouse), médiane de 3 chargements. Le bundling local utilise esbuild (la production utilise Vite/Rollup : les tailles réelles différeront légèrement). Données brutes : `performance/harness-before.json`, `performance/harness-after.json`.

Résultats

Page	Indicateur	Avant	Après	Évolution
Accueil `/`	LCP	8 988 ms	**2 720 ms**	−70 %
	CLS	0	0	=
	JS transféré (gzip)	284 Ko	**87 Ko**	−69 %
	Fin du chargement réseau	8 869 ms	4 161 ms	−53 %
Liste `/evenements`	LCP	14 220 ms	**3 840 ms**	−73 %
	CLS	**0,376**	**0**	corrigé
	JS (gzip)	284 Ko	90 Ko	−68 %

Page	Indicateur	Avant	Après	Évolution
	Images	1 358 Ko	**111 Ko**	−92 %
Connexion `/connexion`	LCP	8 936 ms	3 580 ms	−60 %
	CLS	0	0	=
	JS (gzip)	284 Ko	115 Ko	−60 %
Toutes	Interaction (proxy INP)	72–120 ms	56–120 ms	sous le seuil « bon » de 200 ms

Remarques honnêtes : - Le CSS « après » (15,7 Ko) est plus lourd que « avant » (9,3 Ko) **uniquement à cause du banc**, qui charge d'emblée le CSS de toutes les routes (esbuild n'injecte pas le CSS des modules chargés à la demande ; Vite le fait). En production, chaque route ne charge que son CSS. - La mesure d'images de l'accueil « avant » vaut 0 Ko dans le fichier brut : artefact de mesure (les images PNG n'ont pas été comptées), non retenu. - Avec un LCP de 2,7 à 3,8 s sous forte limitation, l'objectif « bon » (≤ 2,5 s) n'est pas encore atteint dans ces conditions sévères ; il reste à mesurer sur la stack réelle (nginx gzip, cache HTTP, HTTP/2).

Optimisations mises en œuvre

Optimisation	Où
Découpage du code par route + chunks vendeurs séparés (`manualChunks` react / forms)	`frontend/src/App.jsx`, `vite.config.js`
Leaflet et son CSS chargés uniquement sur la carte / la fiche	`components/MapView.jsx`
Images : redimensionnement 1600 px, WebP q78, vignette 640 px, `loading="lazy"`, `decoding="async"`, dimensions explicites, `fetchpriority="high"` pour les 2–3 premières	`middleware/upload.js`, `EventCard.jsx`
Police auto-hébergée (3 fichiers woff2, 69 Ko), `preload`, `font-display: swap`	`index.html`, `tokens.css`
Réservation d'espace (aspect-ratio des cartes, hauteur minimale de `main`) → CLS 0	CSS
Cache : assets hachés `immutable` 1 an, `index.html` `no-cache`, API `Cache-Control: public, max-age=30/60` sur les listes publiques, uploads 30 j	`nginx.conf`, controllers
Compression : gzip nginx + `compression` Express	
Pagination serveur (12 par page, 50 max)	API + UI
Index MongoDB : texte pondéré, `{status,startDate}`, `{status,category,startDate}`, `{venue.city,status}`, 2dsphere, `{organizer,createdAt}`, réservations `{event,status}`, unique partiel	`models/`
Requêtes efficaces : `lean()`, projections, `countDocuments` en parallèle, agrégations pour les statistiques	`repositories/`

Mesures à réaliser sur la stack réelle (commandes)

```
        docker compose up --build -d

        # Lighthouse (4 pages × 3 passages, rapports dans
deliverables/performance/lighthouse)
        cd frontend && npx @lhci/cli@0.14.0 autorun --config=lighthouserc.json
        # ou Chrome DevTools > Lighthouse > mode Navigation, appareil Mobile puis Ordinateur

        # Charge légère API (20 utilisateurs virtuels, 1 min 45) — seuils p95 dans le script
        k6 run --summary-export=perf/results/k6-summary.json perf/load-test.k6.js
```

Pour un vrai « avant / après » Lighthouse : `git checkout bc6dcf9` (version initiale) puis le commit final, mêmes conditions, et reporter les scores et captures dans ce document.